

RESEARCH ARTICLE

ProSt: Provably Secure Homomorphic Steganography

SHAHZAD AHMAD¹ AND STEFAN RASS², (Member, IEEE)¹LIT Secure and Correct Systems Laboratory, Johannes Kepler University Linz, 4040 Linz, Austria

Corresponding author: Shahzad Ahmad (shahzad.ahmad@jku.at)

ABSTRACT This paper introduces Provably Secure Homomorphic Steganography (ProSt), a novel technique that combines the principles of steganography and homomorphic computation. ProSt enables secure computations on data hidden within cover images using steganographic techniques. Unlike traditional steganography, which hides the existence of a message, ProSt assumes the cloud provider knows it is processing hidden data. However, it conceals the specific location and meaning of the secret bits, maintaining their imperceptibility and security. The system utilises the Fredkin gate, a universal reversible logic gate, to perform homomorphic operations on secret bits embedded in image pixels using Least Significant Bit (LSB) steganography. The paper provides an implementation of the scheme, including setup, secret embedding, homomorphic computation (which involves evaluating a single gate that is arbitrarily repeatable), and extraction processes. The security analysis shows that ProSt admits only a cryptographically negligible advantage for an honest-but-curious adversary. This advantage decays with the number of decoy bits and is preserved over any number of gate evaluations. This approach opens new possibilities for secure computation outsourcing in public cloud environments, enabling privacy-preserving data analytics, secure multi-party computation, and confidential image processing on untrusted cloud infrastructure. The paper concludes by discussing potential enhancements and future research directions to further improve the system's security, efficiency, and applicability.

INDEX TERMS Cloud computing, fredkin gate, homomorphic computation, image processing, information security, privacy-preserving computation, secure computation outsourcing, steganography.

I. INTRODUCTION

The proliferation of cloud computing has enabled organisations and individuals to outsource data storage and complex computations on an unprecedented scale. However, this paradigm introduces significant security and privacy challenges: sensitive data processed by untrusted cloud providers is exposed to potential inspection and misuse. While Fully Homomorphic Encryption (FHE) [1] allows for arbitrary computations on encrypted data, it often incurs substantial computational overhead. Another approach involves using hardware-based Trusted Execution Environments (TEEs) [2]. Still, these depend on specific CPU features, require complex

remote attestation protocols, and are not immune to sophisticated side-channel attacks.¹

This paper introduces fully homomorphic steganography (ProSt) as a new primitive for secure computation outsourcing in cloud environments. ProSt leverages steganography not to hide the existence of a secret message but to hide the location of secret data and the nature of the computation itself within a set of cover images. A cloud customer can embed secret data bits into these images and submit them to a public cloud provider for processing. The cloud provider, acting as an honest but curious adversary, executes a specified circuit on the images (processing every pixel uniformly) without ever learning which pixels contain the true secrets or what logical operations are being performed. Since the cloud provider does know about the existence of the data,

The associate editor coordinating the review of this manuscript and approving it for publication was Sedat Akleyek³.

¹<https://github.com/shahzadssg/prost.git>

the typical goal of steganography (*hiding* the existence of an information) is intentionally “given up”, but we can nonetheless prove that its confidentiality guarantee remains intact even if an attacker is aware that the information is there (Theorem 1 proves this formally, Figure 1 shows the intuition). In other words, we combine the confidentiality of steganography (formalised and proven) with the (herein easy) possibility of data processing to construct a provably secure private data processing mechanism.

A. MOTIVATION AND USE CASE

Consider a cloud-based image analytics service. A client (Alice) has an extensive collection of images, each with a sensitive annotation bit (e.g., a proprietary label for a machine learning model or a flag indicating the presence of a person for a privacy filter). The client wants the cloud provider (Carol) to compute a function over these labels, such as aggregating results or running a classifier, without revealing the labels themselves or the classification logic.

ProSt directly addresses this scenario. As shown in Figure 2, the client embeds her secret bits into cover images and uploads them to cloud storage (e.g., Amazon S3). She then provides a circuit description and instructs a cloud computing service (e.g., AWS EC2) to execute it. The cloud provider performs the computation across the entire set of images, obtaining a correct result that is also hidden in the final output image. The client can then download this image and extract the result. Throughout this process, the cloud provider cannot distinguish the secret-carrying pixels from the decoys, nor can it easily deduce the function of the circuit, thus preserving both data and operational privacy.

We stress that the scenario of processing labels should *not* mistakenly be understood as an image classification task; in fact, the “labels” that we let the images steganographically contain need not be related anyhow to the images, nor is there a semantic relation between the label and its carrier image required. The carrier (here: images) is entirely replaceable by anything that allows a small but variable piece of information to be embedded (with images being only natural but not the only examples).

B. GENERAL PROBLEM FORMULATION

Consider the fundamental challenge in privacy-preserving computation on steganographically hidden data. Let $x \in \{0, 1\}^n$ be a secret hidden within one or more steganographic cover images using the embedding function. We wish to compute $f(x)$ where $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is specified as a Boolean circuit composed of logical gates. The critical requirements are:

- 1) The computation must be performed without extracting x from its steganographic embedding
- 2) The result $f(x)$ must remain hidden in steganographic form
- 3) Both the input data and computation results must remain hidden for an honest-but-curious adversary

Traditional approaches would either require extracting the hidden data in a secure environment, performing the computation, and re-embedding the result, or leveraging sophisticated (fully) homomorphic cryptography; neither of which is easy to implement or maintain. Our proposal to use steganography to hide the information eases processing it (since there is no transformation applied to the data), while provably retaining confidentiality by the underlying steganography (Theorem 1).

The paper is organized as follows: Section II reviews related work, Section III presents the ProSt concept, Section IV defines the system model, Section IV-B analyzes adversaries, Section V details the homomorphic steganography, Section VI proves security, Section VII provides a practical example, and Section VIII concludes with future directions.

II. RELATED WORK

Provably Secure Homomorphic Steganography (ProSt) intersects several vital areas in information security, cryptography, and data privacy. By synthesising ideas from these diverse fields, ProSt addresses the growing need for secure and private data processing in scenarios where the content and the existence of sensitive information must be protected. This section explores the relevant prior work that forms the foundation for ProSt, including steganography, homomorphic encryption, secure computation, and information-theoretic security.

Steganography, the art of concealing information within seemingly innocuous carriers, has evolved significantly in the digital age. Fridrich [3] provides a comprehensive overview of steganographic principles, algorithms, and applications in digital media, establishing the foundational concepts upon which ProSt builds. Recent advancements in steganography have focused on improving the undetectability of hidden information. Holub and Fridrich [4] introduced directional filters for designing steganographic distortion, enhancing the ability to embed data with minimal impact on the cover medium. Pevný et al. [5] leveraged high-dimensional image models to achieve highly undetectable data hiding in image steganography. In contrast, Li et al. [6] developed a novel cost function for spatial image steganography, further advancing the field’s capacity for covert communication.

These works have significantly improved the security and efficiency of data-hiding techniques. However, they primarily focus on embedding and extracting static information without considering the possibility of performing computations on the hidden data, a gap that ProSt aims to address.

More recent advancements in provably secure steganography, such as Meteor [7], Discop [8], and public-key steganography based on admissible encoding [9], have focused on achieving strong statistical undetectability and higher capacities within realistic cover distributions. These works significantly advance the field’s capacity for covert communication by meticulously matching cover source

statistics. In contrast, ProSt's security paradigm differs: it assumes the adversary is aware of data being processed but specifically conceals the *location* of secret bits and the *nature of the computation*, rather than hiding the mere existence of information.

While these works have significantly improved the security and efficiency of data-hiding techniques, they primarily focus on embedding and extracting static information without considering the possibility of performing computations on the hidden data. Furthermore, their security metrics often revolve around statistical undetectability (e.g., PSNR, MSE, steganalysis resistance) or maximising embedding capacity, which are not the primary security objectives of ProSt. Our scheme, in contrast, explicitly addresses computing on hidden data while protecting the *location* and *semantic meaning* of secret bits, assuming the adversary is aware of the data's presence.

The concept of homomorphic encryption, which allows computations on encrypted data, serves as a key inspiration for ProSt. Gentry's seminal work [1] introduced the first fully homomorphic encryption scheme using ideal lattices, demonstrating the theoretical possibility of arbitrary computations on encrypted data. This breakthrough opened new avenues for secure computation, though initial constructions could have been more practical for real-world applications. Subsequent research focused on improving the efficiency of homomorphic encryption. Brakerski and Vaikuntanathan [10] proposed a more efficient, fully homomorphic encryption scheme based on the Learning With Errors (LWE) problem, significantly advancing the field's practicality. While these FHE schemes provide powerful tools for secure cloud computation, their high computational overhead and the fact that they produce obviously encrypted ciphertexts can be drawbacks in specific scenarios, motivating our alternative approach.

The practical aspects of implementing homomorphic operations have been extensively studied in the context of encrypted data. Naehrig et al. [11] explored the challenges and potential solutions for making homomorphic encryption practical, addressing issues such as noise growth and computational overhead. Gentry et al. [12] demonstrated the homomorphic evaluation of the AES circuit, showcasing the potential for complex operations on encrypted data. These studies highlight the trade-offs between security, efficiency, and computational capability in homomorphic systems – equally relevant considerations to ProSt. While ProSt operates in the steganographic domain rather than on encrypted data, it faces similar challenges in managing the impact of computations on the hidden information and the cover medium.

The use of reversible computing principles in ProSt, particularly the Fredkin gate, draws from a rich body of research. Perumalla's work [13] comprehensively introduces reversible computing, outlining its principles and potential applications. Saeedi and Markov [14] surveyed techniques

for synthesising and optimising reversible circuits, offering insights into efficient implementations of reversible operations. The Fredkin gate, a fundamental building block in reversible computing, plays a crucial role in ProSt by enabling computations on hidden data while preserving the statistical properties of the cover medium. This approach bridges traditional steganography with the computational capabilities inspired by homomorphic encryption. ProSt relates to the broader fields of secure multi-party computation and privacy-preserving data processing. Lindell's tutorial [15] on simulation-based proofs provides a framework for analysing the security of cryptographic protocols, which we adapt to the steganographic setting. The comprehensive introduction to modern cryptography by Katz and Lindell [16] offers fundamental principles that inform the security considerations in ProSt. In the realm of privacy-preserving data processing, differential privacy, as formalised by Dwork and Roth [17], presents rigorous approaches to protecting individual privacy in statistical databases. While ProSt focuses on hiding the existence of data rather than protecting privacy in released statistics, it shares the goal of enabling practical computations while providing strong security guarantees. Boneh et al. [18] explored private database queries using somewhat homomorphic encryption, demonstrating practical applications of homomorphic techniques. ProSt extends this concept to the steganographic domain, enabling queries and computations on hidden data while maintaining the covertness of the entire operation.

The security analysis of ProSt is grounded in information-theoretic principles. Cachin's seminal work [19] on entropy measures and unconditional security in cryptography, particularly his 'An Information-Theoretic Model for Steganography' [20], provides a framework for analysing the security of steganographic systems. This approach allows for security guarantees that do not rely on computational assumptions, which is crucial when dealing with adversaries that can leverage quantum computing or other super-polynomial computational power.

Dodis's revisiting of Shannon's impossibility results [21] offers insights into the fundamental limits of information-theoretic security. These principles inform the design and analysis of ProSt, helping to establish bounds on the security achievable in various scenarios and guiding the development of provably secure protocols.

Provably Secure Homomorphic Steganography is at the confluence of several critical areas in information security and cryptography. It builds upon advanced steganographic techniques for undetectable data hiding, incorporates concepts from homomorphic encryption and reversible computing to enable secure computations, and draws on secure multi-party computation and information theory principles for its security analysis. By synthesising ideas from these diverse fields, ProSt addresses the growing need for secure and private data processing in scenarios where the content and the existence of sensitive information must be protected.

TABLE 1. Comparison of traditional Steganography, HE, TEEs, and ProSt.

Primitive	Hides Existence of Message	Hides Location of Secret	Computation on Hidden Data	Key Management	Hardware Dependency
Traditional Steganography	Yes	Yes	No	No	No
Homomorphic Encryption	No	N/A	Yes	Yes	No
TEE Computation	No	No	Yes (in enclave)	No	Yes
ProSt	No	Yes	Yes	No (at server)	No

As information security challenges continue to evolve, techniques like ProSt, which combine robust data hiding with secure computation capabilities, are poised to play an increasingly important role in addressing complex privacy and security requirements.

A. COMPARISON TO CLOUD SECURITY PRIMITIVES

As shown in the Table 1, ProSt differs from traditional steganography by not hiding the existence of a computation or hidden data, aligning instead with homomorphic paradigms where the focus is on enabling secure processing while protecting the specifics of the secret (its location and value). ProSt provides a set of trade-offs compared to existing cloud security technologies.

- *Homomorphic Encryption (HE)*: Requires robust key management and is often computationally intensive. ProSt avoids public-key cryptography at the server, relying on a lightweight shared secret (the location map) only between clients.
- *Trusted Execution Environments (TEEs)*: TEEs (e.g., Intel SGX) provide hardware-enforced isolation but introduce dependencies on specific hardware, require complex remote attestation, and have been subject to side-channel attacks that exploit microarchitectural features. ProSt is a purely algorithmic approach, making it hardware-agnostic and resilient to such physical attacks.
- *Traditional Steganography*: Aims to hide the existence of communication, and is indeed a security primitive also for cloud computing [22]. ProSt, in contrast, assumes the communication/computation context is known but hides the data and logic within it.

III. THE SOLUTION CONCEPT

Figure 1 illustrates the concept of how information can be hidden to allow nonetheless processing it, shown in Figure 2. While traditional steganography aims to conceal the existence of a message, ProSt goes a step further by enabling computations on hidden data. Figure 1 shows a secret bit S is hidden within a larger data structure M (e.g., the LSBs of an image) that is filled with random decoy bits. The exact location of S is determined by a secret information π , acting as a key as in symmetric cryptography. This information is only known to Alice. It will later be convenient to consider this secret as acting like a permutation, which determines where Alice will have her secret placed in a carrier vector of

random bits (arranged into a matrix and put into an image). An adversary (Carol), even observing the entire steganogram M , cannot distinguish the true secret from the decoys with any significant advantage. Her posterior knowledge of the secret’s value is only negligibly better than her prior knowledge.

ProSt extends this idea to a computation made across all bits, including all decoys, with a particular focus on the actual secret. In other words, the cloud provider will, in addition to the secret, run the same circuit on all decoy bits, leaving the actual information bit unknown but guaranteed to be under processing too. The secrecy of the result remains intact for its steganographic embedding, without any extra efforts required for processing (unlike homomorphic cryptography, which needs highly nontrivial mechanisms for the same goal). In addition, the use of a universal gate (Fredkin) whose semantics depend on the input values (specifically the control bit input), provides a *heuristic form of circuit privacy*. If the same Fredkin gate-based circuit is executed on different input and control bit configurations, the actual logical operation performed by each gate (e.g., AND, OR) dynamically depends on its control bit, which is itself hidden among decoy bits. While a formal proof of this circuit privacy is a subject for future work, this property heuristically adds to the system’s security without additional computational cost.

A. PRELIMINARIES

We formalise the components of a ProSt scheme starting from the groundwork of traditional steganography.

1) IMAGE REPRESENTATION

Each image in this system is conceptualised as a three-dimensional tensor. This tensor has dimensions of height (h), width (w), and depth (3), corresponding to the spatial dimensions of the image and its color channels, respectively. The height and width represent the number of pixels in the vertical and horizontal directions (we will use a red-green-blue (RGB) color space, but others are admissible via obvious changes). The image I is thus represented as an element of $\mathbb{Z}_{256}^{h \times w \times 3}$. This notation indicates that each element of the tensor is an integer value, typically ranging from 0 to 255, representing the intensity of each color channel for every pixel. The images are available in advance, and the user can choose any one. Moreover, they are related only to the same sizes and formats. To access and manipulate individual pixel values, we write $I[r, c, k]$ to denote an image pixel in row r , column c and color channel k .

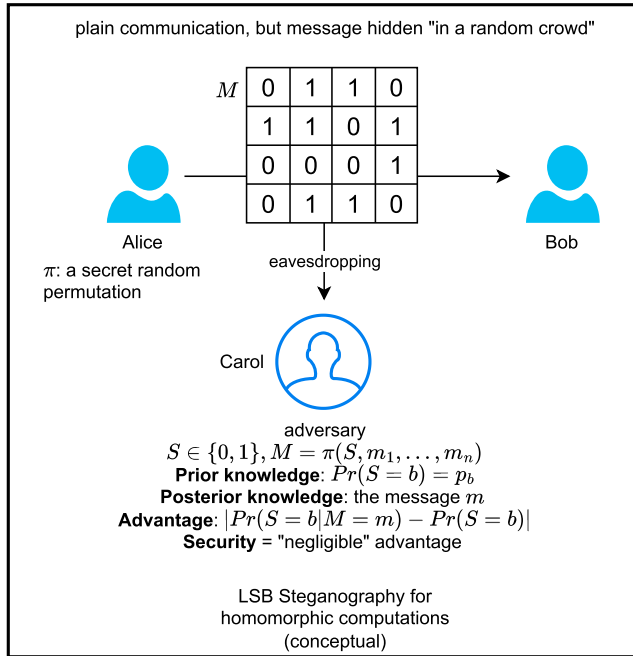


FIGURE 1. Confidentiality from steganography, if the attacker knows that the information "is there", but not "where exactly".

2) LEAST SIGNIFICANT BIT (LSB) STEGANOGRAPHY

We employ LSB steganography primarily due to its simplicity, its ability to allow direct bit-level manipulation for homomorphic computation, and its inherent visual imperceptibility, which ensures the cover images remain seemingly intact despite containing modified bits.

In our ProSt scheme, we employ this LSB steganography technique to embed secret bits into specific pixels of the cover images, facilitating subsequent homomorphic computations on the concealed data while maintaining visual imperceptibility. To *embed* a secret bit b at a confidential position (r, c, k) in an image, we will put $lsb(I[r, c, k]) \leftarrow b$, where lsb is the least-significant bit. We denote this operation as $Emb(I, b, (r, c, k))$. Similarly, the *extraction* is a function $Ext(I, (r, c, k))$ that returns the $lsb(I[r, c, k])$ of the image I at the given position (r, c) and color channel k .

3) COMPUTATIONS ON HIDDEN DATA

The significant difficulty of FHE is the fact that the information is not available for processing in plain, but instead has undergone some mathematical transformation (encryption). In steganography, however, the information is there "in plain", and *could* be processed directly, but we need to hide its exact location to avoid disclosure. Towards keeping the secret information hidden, we will process *all* data bits in parallel, knowing that the actual secret is among them, but not at which position. Since steganography does not generally transform the secret data,² it remains processable

²we assume that it is not combined with encryption, such as some implementations like OpenStego [23] offer.

under the guise of the steganogram, securely, as Theorem 1 asserts. This raises the question of how well (quantitatively) a secret is "hidden" among many other bits. Clearly, this hiding depends on the number of bits available to hide the secret. Obviously, a single bit cannot remain secret (without encryption; see the above part of Figure 1), but within a set of other bits, the secret can remain relatively "secure".

The Fredkin gate maps a triple of input bits to a triple of output bits by a function $F : \{0, 1\}^3 \rightarrow \{0, 1\}^3$, defined as:

$$F(c, x, y) = \begin{cases} (c, x, y) & \text{if } c = 0 \\ (c, y, x) & \text{if } c = 1 \end{cases}$$

where c is the control bit, and x and y are the input bits.

It is well known that this gate is universal (i.e., can compute AND, OR and NOT operations), and an inspection of the truth table immediately shows the following conservation property that we require for later reference:

Lemma 1: [Hamming weight invariance] Let $w \in \{0, 1\}^3$ be a triple of bits, and let $F(w)$ be the output of a Fredkin gate. Then, the Hamming weight of w and $F(w)$ is the same.

The value of Lemma 1 will lie in the preservation of the distribution of 0 and 1 bits across the data (least significant bit regions), over any computation using a Boolean circuit, if this circuit is using only Fredkin gates. Since these are universal, we may assume an all-Fredkin-gate circuit without loss of generality. For a Fredkin gate $F(c, x, y) = (c, x', y')$, the total number of 1s in the input equals the total number of 1s in the output (i.e., $c + x + y = c + x' + y'$). This follows from the gate's reversibility: if $c = 0$, $(x', y') = (x, y)$; if $c = 1$, $(x', y') = (y, x)$.

The LSB embedding, together with a pixel-by-pixel computation (parallelised), allows for evaluating any Boolean circuit across the entire image, as indicated in Figure 2 for a single gate. It then remains to show that the hiding of the data does not deteriorate along (longer chains of) computations, but the preservation of distributions that Lemma 1 asserts will preserve security here.

EXTENSION TO IMAGE OPERATIONS

We extend the Fredkin gate operation to work on entire images. Let I_1, I_2, I_c be cover images to hide the secret inputs b_1, b_2 and control bit c . We extend the Fredkin operation $F : \mathbb{Z}_{256}^{h \times w \times 3} \times \mathbb{Z}_{256}^{h \times w \times 3} \times \mathbb{Z}_{256}^{h \times w \times 3} \rightarrow \mathbb{Z}_{256}^{h \times w \times 3} \times \mathbb{Z}_{256}^{h \times w \times 3} \times \mathbb{Z}_{256}^{h \times w \times 3}$ as: $F(I_c, I_1, I_2) = (O_c, O_1, O_2)$ where for each pixel at (r, c) and channel k :

$$(lsb(O_c[r, c, k]), lsb(O_1[r, c, k]), lsb(O_2[r, c, k])) \leftarrow F(lsb(I_c[r, c, k]), lsb(I_1[r, c, k]), lsb(I_2[r, c, k])) \quad (1)$$

This formulation applies the Fredkin gate operation to each corresponding pixel and channel across the three images, using the LSB of the control image as the control bit. It is important to remark that the positions of the inputs in each image are the same as where the output bits are placed. This assures that (i) Alice (or whoever receives the final data and

knows where the inputs have been placed) will be able to recover the final results after the processing, and (ii) there will be no interference with the parallel processing of decoy bits.

IV. SYSTEM AND THREAT MODEL

In this section, we will see the system model and the formal definition of ProSt. The protocol involves the following primary entities:

- **Alice (Sender):** The party who wishes to perform a computation on secret data. Alice embeds her secret bits into cover images at a secret position and specifies a Boolean circuit to be evaluated.
- **Carol (Untrusted Server or Cloud Provider):** A compute server that receives the cover images and circuit description from Alice, applies the specified operations, and returns the resulting images. Carol is assumed to be honest-but-curious (HBC), meaning she follows the protocol correctly but may attempt to extract information from the images and operations she processes.

The intended recipient of the computation's result may be Alice herself or a receiver party (e.g., Bob) who shares the secret position key with Alice. Importantly, Carol is not the recipient of the hidden message; her role is computational, not communicative. This distinction resolves the apparent contradiction in traditional steganography, where the receiver must be unaware of a message's existence. In ProSt, the steganographic component hides which bits among many are secret, not that a computation is occurring.

A. FORMAL DEFINITION OF ProSt

ProSt is defined as a tuple of four algorithms: $\text{ProSt} = (\text{LocGen}, \text{Emb}, \text{Comp}, \text{Ext})$, operating over a domain of cover images I (e.g., $h \times w \times 3$, for images with 3 colors like RGB) and secret bit b :

- i) $\text{LocGen}(h, w, 3)$: Generates a secret position (r, c, k) where $0 \leq r < h$, $0 \leq c < w$, and $0 \leq k < 3$ (channel index), chosen uniformly at random from $h \cdot w \cdot 3$ possibilities. This position is shared only between Alice and the intended recipient, e.g., Bob. The exchange of this secret is doable by standard cryptographic techniques (key exchange protocols from which positions can be derived, up to sending (r, c, k) in an encrypted message).
- ii) $I' \leftarrow \text{Emb}(I, b, (r, c, k))$: Embeds the secret bit b into a cover image I at position (r, c, k) using least significant bit (LSB) substitution. Returns the modified images I' . This is done by Alice.
- iii) $(O_c, O_1, O_2) \leftarrow \text{Comp}(G, (I_c, I_1, I_2))$: Evaluates a Fredkin gate G with input images I_c, I_1, I_2 carrying the control bits, and with outputs to be embedded in images O_c, O_1, O_2 . The algorithm Comp applies each gate pixel-wise across all positions in the images (see Section VII-C for an implementation). This is done

by the (potentially distrusted) third party (e.g., cloud provider).

- iv) $b \leftarrow \text{Ext}(I, (r, c, k))$: Extracts the resulting secret bits from the output image I at position (r, c, k) . This is done by the recipient of the final outputs (can be Alice herself, or also Bob).

The circuit C is constructed from Fredkin gates, which are reversible and universal, enabling any Boolean function to be computed homomorphically on the hidden bits. The use of LSB embedding ensures imperceptibility, while the uniform application of gates across all pixels prevents Carol from distinguishing the secret position.

ProSt departs from traditional steganography by assuming Carol knows she is processing data, aligning with homomorphic computation paradigms. The security relies on the entropy of the secret position: with $n = h \cdot w \cdot 3$ possible positions (e.g., $128 \times 128 \times 3 = 49,152$ for a small image), an adversary's advantage in guessing (r, c, k) is negligible ($\Theta(1/n)$, per Section VI). This makes ProSt suitable for untrusted environments where encrypted traffic is common, but intermediate decryption poses risks.

B. ADVERSARY MODEL AND SECURITY DEFINITION

We formalise the security goals of ProSt under an honest-but-curious adversarial model, consistent with standard cryptographic definitions (e.g., [24]). The adversary is computationally unbounded but constrained in capabilities as follows:

Adversary's Knowledge. The adversary $\mathcal{A}$ knows:

- All public cover images, including the LSBs of all pixels;
- The Boolean circuit C (in gate-level reversible form);
- The distributional prior on the secret input bits $S_1, \dots, S_\ell$ (e.g., uniform or Bernoulli);
- The steganographic mechanism (e.g., LSB embedding strategy).

Adversary's Limitations. $\mathcal{A}$ does **not**:

- know of the secret positions $\{(r_i, c_i, k_i)\}$ of the embedded bits;
- mount active attacks, with or without interaction with the honest party (e.g., manipulations of images during processing, or having Alice embed or extract secrets in chosen cover images);
- Access to multiple stego images embedding the same secret data.

This corresponds to a *non-interactive, single-shot* security model, which is common in steganographic and watermarking applications. The attacker's goal is to extract or infer any new information about the embedded secrets.

We will adopt a cryptographically inspired asymptotic security definition, analysing the attacker's "advantage" (formalised later as Definition 1) in terms of the number n of possible locations where the secret could be embedded (this corresponds to a security parameter in usual cryptographic notions, and quantifies the size of the search space for

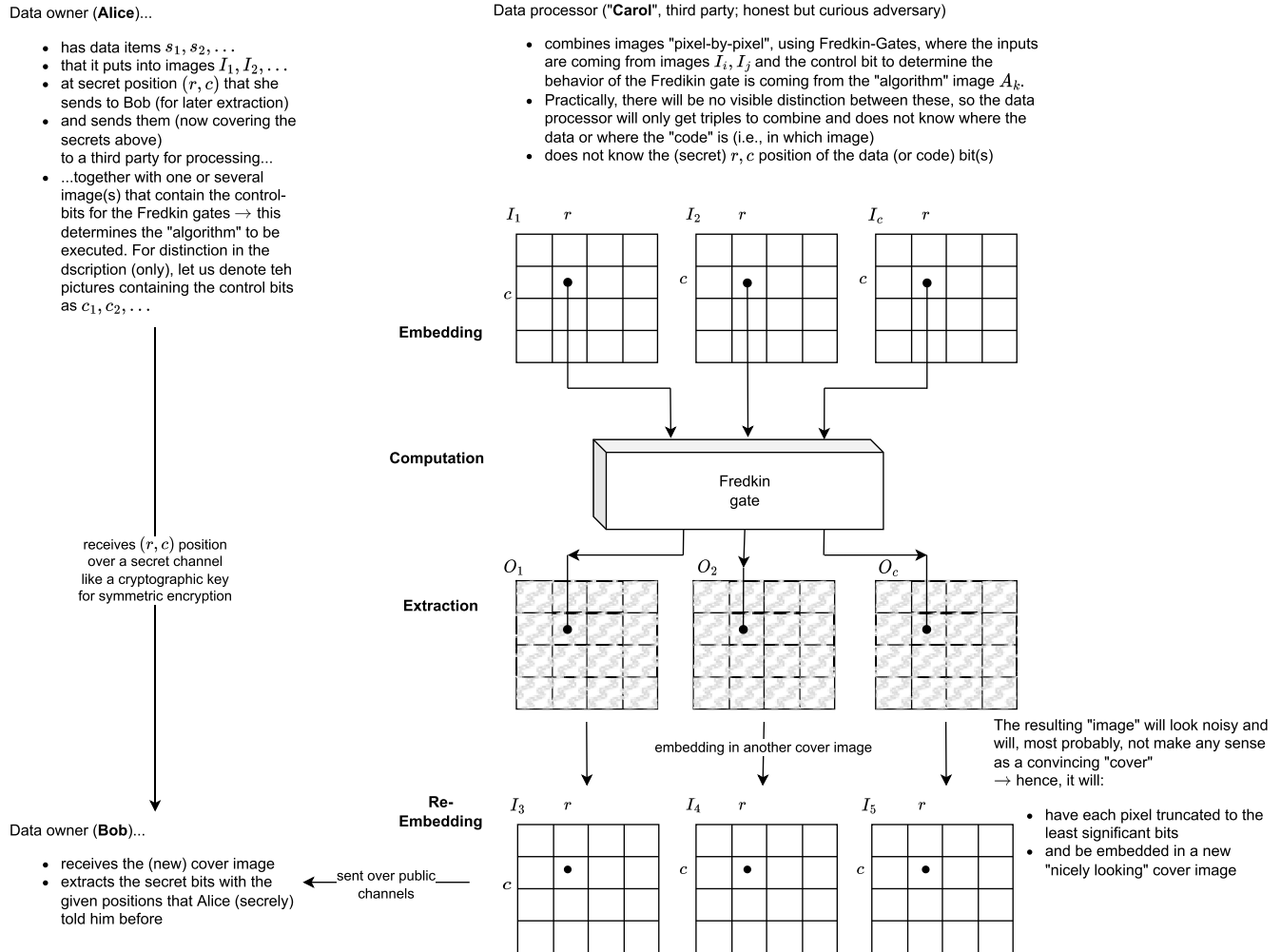


FIGURE 2. Conceptual illustration of provably secure homomorphic Steganography (for brevity, we have not shown computation on all the elements and used a fixed (arbitrary) color channel).

brute-force attacks). Theorem 1 will exhibit the attacker's advantage to be negligible, in the rigorous sense defined below.

C. GAME-BASED SECURITY DEFINITION

ProSt aims to protect the confidentiality of secret bits embedded in cover images during computation by an untrusted cloud provider. Unlike traditional steganography, where the goal is to hide the existence of a message, ProSt assumes the cloud provider (Carol) knows she is processing data as part of a computational task but cannot determine *where* the secret bits are embedded or *what* computation is being performed. The security of ProSt thus hinges on the indistinguishability of the secret position (r, c, k) among all possible positions in the cover images. We stress that this is essentially the same as in symmetric cryptography, since the position of the secret plays the role of a secret key. We formalize this through the following security game between a challenger (representing Alice) and an adversary

$\mathcal{A}$ (representing Carol or an eavesdropper), displayed in Figure 3.

To quantify the chances of "winning" the security game $\text{Game}_{\text{ProSt}}^{\mathcal{A}}(n)$, let the secret be b (fixed), and consider the attacker's final guessing step in the game. The chances to output $S = b$ are conditional on the attacker's information m , which is the steganogram. Hence, a correct guess in step 3 of the game will occur with probability $\Pr(S = b | M = m)$. This chance cannot be less than the chances to guess correctly, independent of any steganogram, which is (based on the attacker's prior knowledge) the unconditional likelihood $\Pr(S = b)$. We thus define security in the usual way, requiring only a negligible difference due to the use of ProSt. We call a function $v : \mathbb{N} \rightarrow \mathbb{R}^+$ *negligible*, if for every $c > 0$ there is an integer $n_0 \in \mathbb{N}$ such that $v(n) \leq n^{-c}$ for all $n \geq n_0$ (i.e., v decays faster than any polynomial).

Definition 1: [Advantage, Security] Let $S \sim \text{Ber}(p_0)$ be the (Bernoulli)-distributed secret with $\Pr(S = 0) = p_0$, $\Pr(S = 1) = p_1 = 1 - p_0$, and let the attacker's

Fix the system parameters, including the size (height h , width w and color channels, e.g. 3) of the cover images. We assume this to be common knowledge of both parties. Let the number $2 \leq n \in \mathbb{N}$ of cover images be variable.

Game_{ProSt}^A(n) entails the following steps

- 1) **Setup:** The challenger selects parameters $h, w, 3$ (image dimensions and channels, e.g., $128 \times 128 \times 3$) and a secret bit b .
- 2) **Embedding:** The challenger runs $\text{LocGen}(h, w, 3)$ to choose a secret position (r, c, k) uniformly at random from $n = h \cdot w \cdot 3$ possibilities, and $\text{Emb}(I, b, (r, c, k))$ to embed b into a chosen cover images I , producing I' .
- 3) **Challenge:** The adversary $\mathcal{A}$ receives I' and guesses a location (r', c', k') and extracts the candidate secret b' from it.
- 4) **Outcome:** $\mathcal{A}$ wins if and only if $b' = b$.

FIGURE 3. ProSt security game.

information be $\Pr(S = b) = p_b$ for $b \in \{0, 1\}$. The attacker's advantage in **Game_{ProSt}^A** is $\text{Adv}_{\mathcal{A}} := |\Pr(S = b|M = m) - \Pr(S = b)|$, and we call the steganographic systems *secure*, if $\text{Adv}_{\mathcal{A}} \leq \nu(n)$, where ν is a negligible function.

In the special case of a uniform distribution of the secret bit, we have $\Pr(S = b) = \frac{1}{2}$, and the advantage takes the more often seen form $\text{Adv}_{\mathcal{A}} := |\Pr(\mathcal{A} \text{ wins}) - \frac{1}{2}|$, which resembles semantic security of encryption (of single bits), if the advantage shall be negligible. We will adhere to the more general definition and let the adversary's prior knowledge be arbitrary.

V. OUR ProSt SCHEME

We now describe the process by which ProSt embeds bits and executes a Boolean circuit. Let C be a Boolean circuit, which is acyclic and composed of a Fredkin gate, with ℓ input wires and one output wire.

A. INPUT EMBEDDING

To begin, a set of cover images is allocated to serve as wires. Specifically, Alice selects N cover images $I_1, \dots, I_N$, each of size $h \times w$ with three color channels. She designates ℓ of these as input wires, with the remainder serving as ancillary or output wires. Using a keyed location generator $\text{LocGen}(h, w, 3)$, Alice selects, for each input wire $i = 1, \dots, \ell$, a secret pixel position (r_i, c_i) and a color channel $k_i \in \{1, 2, 3\}$ uniformly at random. Each secret bit $b_i \in \{0, 1\}$ is embedded by substituting the least significant bit (LSB) of $I_i[r_i, c_i, k_i]$ with b_i , while leaving all other LSBs in the image as independent random "decoy" bits. Consequently, each secret bit is concealed among $n = h \times w \times 3 - 1$ random LSBs (see Figure 2). Additionally, Alice selects a position

(r_N, c_N, k_N) in the image designated for the final output wire and keeps it secret for use in extraction.

B. CIRCUIT DESCRIPTION

Alice constructs a public description of the circuit C : she assigns each wire name in C an index in $\{1, \dots, N\}$. Equivalently, C is expressed as a sequence of N_g gates $\{G_i\}_{i=1}^{N_g}$, where each gate G_i is of the form (Fredkin; $i_c, i_1, i_2 \rightarrow o_c, o_1, o_2$). Here, $i_c, i_1, i_2 \in \{1, \dots, N\}$ are the indices of the cover images for the three input wires of G_i , and $o_c, o_1, o_2 \in \{1, \dots, N\}$ are the indices for the three output wires. Alice sends the set of stego (cover) images and the circuit description C to the cloud provider over a public channel. As C is acyclic, there exists a valid evaluation order $G_1, \dots, G_{N_g}$ (the topological graph order). To ease the practicalities, we define a domain-specific language in Section VII-A to specify the circuit as required.

C. DATA PROCESSING IN PRIVACY

The cloud provider, operating in an honest-but-curious model, evaluates C as follows: for each gate G_i in sequence, it reads the LSBs of images $I_{i_c}[r, c, k]$, $I_{i_1}[r, c, k]$, and $I_{i_2}[r, c, k]$ at every pixel location (r, c, k) , treating cover images as 3D arrays of pixels. The cloud provider applies the Fredkin gate to these three bits in parallel and across all pixels. The resulting three output bits are written into the LSBs of the images $I_{o_c}, I_{o_1}, I_{o_2}$ at the same (r, c, k) coordinates, as specified by the circuit (see Figure 2). Gates are thus applied pixel-wise in parallel. Crucially, the output wires $I_{o_c}, I_{o_1}, I_{o_2}$ are chosen to avoid overwriting any secret-bearing pixel in the input images. In our domain-specific language (DSL) and compiler, each wire is bound to a distinct image or image channel, ensuring that gate outputs are written to separate cover images or unused channels, thereby preserving all hidden LSBs.

D. RESULT EXTRACTION

After all gates are processed, Bob (or Alice) performs the stego extraction $\text{Ext}(I_N, (r_N, c_N, k_N))$ on the final output image I_N at the known secret location to recover the single output bit.

VI. SECURITY ANALYSIS

ProSt's security analysis differs from traditional steganography's focus on imperceptibility or capacity. Here, we analyse the adversary's advantage in identifying the *true secret bit's value* given awareness of the steganogram, emphasising location confidentiality rather than statistical undetectability. In this section, we analyse the security of the ProSt system by considering three distinct strategies that an adversary might exploit to guess the secret bit S . These cover the adversary's potential strategies and the corresponding probabilities of success; security is understood here as the attacker's advantage over trivial strategies (as usual in cryptography).

A. GUESSING THE SECRET DIRECTLY (PRIOR KNOWLEDGE)

The first case considers the adversary's ability to guess the secret bit S directly based on their prior knowledge of its distribution. Let S follow a Bernoulli distribution with $\Pr(S = 0) = p_0$ and $\Pr(S = 1) = p_1 = 1 - p_0$. In this scenario, the adversary's success probability is determined solely by these prior probabilities, i.e., $\Pr(S)$. This represents the baseline level of knowledge the adversary possesses before observing any steganographic communication or processing. Importantly, this case does not depend on the embedding process or the number of decoy bits. Thus, the adversary's success rate in this case is limited to their prior knowledge and does not exploit any additional information from the steganogram.

B. GUESSING THE SECRET BY POSITION

The second case examines the adversary's ability to guess the secret by identifying its embedding position (r, c, k) within the image. The search space is defined by the dimensions of the image, where h is the height, w is the width, and 3 represents the number of color channels (e.g., RGB). The probability of correctly guessing the secret's position is given by:

$$\Pr(\text{Correct Position}) = \frac{1}{h \cdot w \cdot 3}.$$

This probability depends on the size of the carrier images and is constrained by the image dimensions. For smaller images or limited embedding spaces, the adversary's chances of correctly guessing the position increase, thereby reducing the system's security. This highlights the importance of using sufficiently large cover images to reduce the adversary's probability of success in this case. However, this strategy only pays off if the chances to guess the position are higher than the chances to guess the secret from prior knowledge ($\Pr(S)$; see the previous paragraph).

C. ADVANTAGE FROM HAVING THE STEGANOGRAM

In light of (at least) the two previous strategies, let us now consider the general advantage gained by the attacker from the knowledge of the steganogram whenever $\Pr(S = b)$ is less than $1/(h \cdot w \cdot 3)$, we may assume (w.l.o.g.) that the pictures are large enough to make guessing the location (much) less likely than guessing the bit directly from prior knowledge. This is not difficult if we draw the decoy bits from a distribution that resembles the attacker's presumed prior knowledge so that taking a random draw from the (urn) of $h \cdot w \cdot 3$ many bits will give a (random) bit x equal to the actual secret, with probability $\Pr(S = x)$. Thus, we are interested in how much the attacker gets beyond the prior knowledge $\Pr(S = b)$.

Some information may be gained by looking for correlations among the bits processed. Still, since our scheme processes all bits (the actual secrets and decoy bits) in parallel and under the same regime, correlations would appear between the decoy bits as well as the actual information

bits (although they undergo different processing due to the (random) determination of the Fredkin gates by the control bits). Correlations are thus already and fully inferable from the circuit description (that the data processor necessarily knows). Still, since the circuit is independent of the data, it cannot (information-theoretically) provide any additional information about the inputs.

We first prove that embedding a bit inside a steganogram of size n makes it nearly impossible for an attacker to discover the secret correctly. To model the process of embedding a bit at an unknown location, we consider the embedding operation Emb as just putting a bit into a set of independent random bits, i.e., we let Emb put the secret b into the least significant bit of a specified pixel, and allowing the "decoy" bits be the LSBs of all other pixels inside the same image (see Figure 1 for an illustration, where the matrix inside which the secret is located is made from the LSBs of all pixels, taking an extremely small image of size 4×4 only, giving $n = 4 \cdot 4 = 16$). The number n , in the following, is thus the total number of pixels, and as such, relatively easy to scale up (towards asymptotic security guarantees).

Theorem 1 (Information Hiding): Let $0 < p_0 < 1$, and let $S \sim \text{Ber}(p_0)$ be a secret bit. Let M be a steganogram that hides the secret S in a vector $M = (M_0, M_1, \dots, M_{n-1})$ of size n . The secret S is placed at an unknown position j , chosen uniformly at random from $\{0, \dots, n-1\}$. All other $n-1$ positions $i \neq j$ are filled with independent random decoy values $M_i \sim \text{Ber}(p_0)$. Then, this steganogram is secure in the sense of Definition 1 with

$$\text{Adv}_{\mathcal{A}} \in \Theta\left(\frac{1}{n}\right)$$

Proof: Let $S = b$ be the specific value of the secret bit, which occurs with probability $\Pr(S = b) = p_b$. Let $\mathcal{L}$ be the random variable representing the uniformly chosen position of the secret bit, so $\Pr(\mathcal{L} = j) = 1/n$ for any $j \in \{0, \dots, n-1\}$. The adversary observes the steganogram $M = (M_0, M_1, \dots, M_{n-1})$ and tries to infer the value of S . We are interested in the posterior probability $\Pr(S = b | M = m)$, where $m = (m_0, m_1, \dots, m_{n-1})$ is the observed steganogram.

By definition,

$$\Pr(S = b | M = m) = \frac{\Pr(S = b, M = m)}{\Pr(M = m)}$$

Since the adversary is assumed to extract from a specific position, say M_0 , we can simplify the problem by considering the event $(S = b, M_0 = b)$ and normalize by $\Pr(M_0 = b)$.

First, let's calculate $\Pr(S = b, M_0 = b)$. This event can occur in two mutually exclusive ways:

- The secret S is at position M_0 , and its value is b .** This implies that the randomly chosen position $\mathcal{L}$ for the secret is 0. The probability of this is $\Pr(\mathcal{L} = 0) = 1/n$. Given that $\mathcal{L} = 0$, the bit M_0 is the secret S . So, $M_0 = b$ is equivalent to $S = b$. The probability of this sub-event

is:

$$\Pr(\mathcal{L} = 0 \wedge S = b) = \Pr(\mathcal{L} = 0) \cdot \Pr(S = b) = \frac{1}{n} \cdot p_b$$

- b) **The secret S is at a position other than M_0 , and M_0 (which is a decoy) also happens to be b , AND the secret S itself has value b .**

The probability that the secret is *not* at position 0 is $\Pr(\mathcal{L} \neq 0) = 1 - 1/n = \frac{n-1}{n}$. In this case, M_0 is a decoy bit, which is drawn independently from $\text{Ber}(p_0)$, so $\Pr(M_0 = b | \mathcal{L} \neq 0) = p_b$. The secret S is drawn from $\text{Ber}(p_0)$, so $\Pr(S = b | \mathcal{L} \neq 0) = p_b$. Since M_0 (as a decoy) and S are independent when S is not at M_0 , the probability of this sub-event is:

$$\begin{aligned} \Pr(\mathcal{L} \neq 0 \wedge S = b \wedge M_0 = b) &= \Pr(\mathcal{L} \neq 0) \cdot \Pr(S = b) \\ &\quad \cdot \Pr(M_0 = b) \\ &= \frac{n-1}{n} \cdot p_b \cdot p_b \\ &= \frac{n-1}{n} p_b^2 \end{aligned}$$

Summing these two mutually exclusive probabilities, we get:

$$\Pr(S = b, M_0 = b) = \frac{1}{n} p_b + \frac{n-1}{n} p_b^2$$

Next, we need the denominator $\Pr(M_0 = b)$. Since the position M_0 is one of n positions, chosen uniformly at random, M_0 could either be the secret (with probability $1/n$) or a decoy (with probability $\frac{n-1}{n}$). In either case, the value $M_0 = b$ has probability p_b . Therefore, $\Pr(M_0 = b) = p_b$.

Now we can compute the posterior probability:

$$\begin{aligned} \Pr(S = b | M = m) &= \frac{\Pr(S = b, M_0 = b)}{\Pr(M_0 = b)} \\ &= \frac{\frac{1}{n} p_b + \frac{n-1}{n} p_b^2}{p_b} \\ &= \frac{1}{n} + \frac{n-1}{n} p_b \\ &= \frac{1}{n} + p_b - \frac{p_b}{n} \\ &= p_b + \frac{1-p_b}{n} \end{aligned}$$

The adversary's advantage, as per Definition 1, is:

$$\begin{aligned} \text{Adv}_{\mathcal{A}} &= |\Pr(S = b | M = m) - \Pr(S = b)| \\ \text{Adv}_{\mathcal{A}} &= \left| \left(p_b + \frac{1-p_b}{n} \right) - p_b \right| \\ \text{Adv}_{\mathcal{A}} &= \left| \frac{1-p_b}{n} \right| \end{aligned}$$

Since $0 < p_b < 1$ (otherwise the secret would be trivial or fixed, making hiding irrelevant), we have $0 < (1 - p_b) < 1$. Thus, for $n > 1$, the advantage is positive:

$$\text{Adv}_{\mathcal{A}} = \frac{1-p_b}{n}$$

TABLE 2. Recommended security parameters for ProSt.

Configuration	n (positions)	Adv. Θ	Security Level
$128 \times 128 \times 3$	49,152	2.0×10^{-5}	Minimal
$512 \times 512 \times 3$	786,432	1.3×10^{-6}	Moderate
$1024 \times 1024 \times 3$	3,145,728	3.2×10^{-7}	Good
$10 \times 512 \times 512 \times 3$	7,864,320	1.3×10^{-7}	High
$4 \times 1024 \times 1024 \times 3$	12,582,912	7.9×10^{-8}	Very High

This advantage clearly falls within $\Theta(1/n)$, as $\lim_{n \rightarrow \infty} \frac{(1-p_b)/n}{1/n} = 1 - p_b$, which is a constant in $(0, 1)$. $\square$

Remark 1 (Implications for Negligibility and Practical Security): The advantage of $\Theta(1/n)$ derived in Theorem 1 means that the adversary's probability of success in guessing the secret bit S is negligibly better than their prior knowledge $\Pr(S = b)$. However, it is crucial to clarify the definition of "negligible" in this context.

In computational complexity theory and modern cryptography, a function $v(n)$ is considered "negligible" if it decays faster than any inverse polynomial in n (i.e., for every positive constant c , $v(n) < n^{-c}$ for sufficiently large n). A function like $1/n$ is itself an inverse polynomial (n^{-1}) and therefore does not satisfy this strict definition of cryptographic negligibility. Instead, $1/n$ is often referred to as "statistically negligible" or "polynomially small."

1) PRACTICAL SECURITY REQUIREMENTS

For ProSt to achieve cryptographic negligibility in the strictest sense, the advantage would need to decay super-polynomially fast (e.g., $1/2^n$ or $1/n!$). The $\Theta(1/n)$ bound implies that if n is only polynomially large (e.g., related to the size of a standard image), a computationally unbounded adversary could potentially gain a non-negligible advantage.

2) RECOMMENDED SECURITY PARAMETERS

For practical deployments against honest-but-curious adversaries, we recommend maintaining $n \geq 2^{20} \approx 10^6$ positions to achieve adversary advantage $\Theta < 10^{-6}$. This can be realized through:

- **High-resolution images:** A single $1024 \times 1024 \times 3$ RGB image provides $n \approx 3.1 \times 10^6$
- **Multiple images:** Ten $512 \times 512 \times 3$ images collectively provide $n \approx 7.9 \times 10^6$
- **Combined approach:** Four $1024 \times 1024 \times 3$ images yield $n \approx 1.26 \times 10^7$ with advantage $\Theta < 8.0 \times 10^{-8}$

Table 2 summarizes recommended configurations. When discussing "provably secure," we explicitly state that the security here refers to a statistically negligible advantage, which is strong for an honest-but-curious adversary in practice, especially if n can be made sufficiently large (e.g., by spreading data across many large images or leveraging exceptionally high-resolution images). The security parameter n (number of possible embedding locations) must be very large to provide adequate practical security.

Corollary 1 (Security Under Computation): Let a circuit C be given, consisting of N gates, all of which are Fredkin gates. Furthermore, let the inputs of the circuit be hidden steganographically in the sense that Theorem 1 describes. Then, the hiding property as asserted by Theorem 1 is preserved across the circuit evaluation.

Proof: This is a simple induction argument over the number N of gates. The induction base case ($N = 0$) is covered by Theorem 1, where the advantage is $\Theta(1/n)$.

For the induction step, assume that after evaluating $N - 1$ Fredkin gates, the distribution of the LSBs across all pixels in the images (including the secret bits and decoy bits) remains consistent with the setup described in Theorem 1. Specifically, the distribution of 0s and 1s among the bits that *could potentially be* the secret (or are currently holding intermediate secret results) is maintained.

Consider the evaluation of the N -th Fredkin gate. By Lemma 1 (Hamming weight invariance), a Fredkin gate $F(c, x, y) = (c, x', y')$ preserves the total number of 1s (and 0s) in its input triplet: $c + x + y = c + x' + y'$. This property is applied pixel-wise to the LSBs of the input images.

Since the Fredkin gate preserves the aggregate distribution of 0s and 1s for *every* triplet of pixels it operates on, it implies that the *overall* distribution of 0s and 1s within the LSB layers of the images is statistically preserved. The operation Comp processes all pixels uniformly, meaning the actual secret bit (which is located at a secret position j) and all decoy bits are subjected to the same logical operation based on their corresponding pixel values. The output bit at the secret position j will be the result of the desired computation, still hidden among $n - 1$ other decoy bits (the outputs of the same Fredkin gate applied to decoy inputs).

The core assumption of Theorem 1 is that the secret is a single bit hidden among $n - 1$ independent random decoy bits. The Fredkin gate, by maintaining the bit distribution locally, ensures that this overall structure is maintained. No information is leaked about the secret's specific position or value from the output images that was not already available in the input images (apart from the $O(1/n)$ statistical advantage). The transformation of bits (secret and decoy) is consistent and uniform across the images.

Thus, the conditions of Theorem 1 remain satisfied after any number of Fredkin gate evaluations, and the negligibility of the attacker's advantage, here $\Theta(1/n)$, is preserved. The number of additional cover images required for intermediate results is at most $O(N)$ for a circuit with N gates (as each Fredkin gate involves 3 input and 3 output wires, necessitating temporary storage for outputs, which can be reused). $\square$

D. SIDE-CHANNEL ATTACKS

In a cloud context, side-channel attacks are a primary concern. ProSt is designed to be resilient against several classes of such attacks:

- **Timing Attacks:** Since all gate operations are applied uniformly pixel-wise across the entire image(s), the execution time is independent of the data values or the secret's location. This homogeneity prevents timing variations from leaking information.
- **Memory Access Attacks:** The protocol requires Carol to read from and write to the entire memory region corresponding to the input and output images for each gate evaluation. This uniform access pattern prevents an adversary from inferring secret locations by monitoring memory or cache access patterns.
- **Hardware-Specific Attacks:** Unlike TEEs, ProSt is purely algorithmic and does not rely on specialised hardware features. It is therefore not vulnerable to microarchitectural attacks (e.g., Spectre, Meltdown) or physical side-channels targeting CPU enclaves.

E. PARTIAL HIDING OF COMPUTATION LOGIC

An additional security feature implied by the use of Fredkin gates in ProSt is the *heuristic* hiding of the circuit's logic from the cloud provider. While Carol receives the circuit's topology (the graph of gates), the specific function of each Fredkin gate remains ambiguous. This is because a gate's function is determined by its control input, which is itself a secret bit hidden in a cover image.

For instance, by setting a control input i_c to 0 or 1, a Fredkin gate can be configured to perform an AND or OR operation, respectively. Without knowing the value of the secret bit controlling the gate, the adversary cannot distinguish between these fundamental operations. This property, combined with obfuscation techniques like adding dummy gates and wires (see Section VII), makes it difficult for the cloud provider to reverse-engineer the client's proprietary algorithm.

Important Caveat – Heuristic Security Only: We emphasize that circuit privacy in ProSt is *heuristic* and does *not* provide formal information-theoretic or computational guarantees. The obfuscation techniques (dummy gates, wire renaming, randomized gate ordering) provide practical obscurity but do not guarantee provable hiding of the circuit's function. Specifically:

- The circuit topology is known to Carol
- Statistical analysis of gate frequencies could leak information
- Multiple executions on different inputs could enable circuit reconstruction
- A formal security definition for circuit privacy is lacking

The total advantage an adversary has in guessing the circuit's function is bounded by $N \cdot \Theta\left(\frac{1}{n}\right)$ for a circuit with N gates and n decoy bits, which remains statistically negligible for large n . However, a formal proof of *computational circuit privacy* is left for future work.

For applications requiring provable circuit privacy, we recommend combining ProSt with additional circuit obfuscation techniques from the homomorphic encryption literature or

using ProSt in scenarios where the computation itself is not sensitive (only the data is).

VII. PRACTICAL IMPLEMENTATION

In this section, we provide a concrete illustration of ProSt. We present a Python implementation that demonstrates the embedding of secret bits into cover images, the homomorphic evaluation of a Boolean circuit using Fredkin gates, and the extraction of the computation result. This example aligns with the theoretical constructs outlined in Sections III–V, showcasing how ProSt enables secure computation on hidden data in an untrusted environment.

To allow circuit specifications in the familiar form of using traditional Boolean gates (AND, OR, NOT), we implemented a domain-specific language (DSL) to specify the circuit in text-based form, along with a parser and interpreter to evaluate the circuit using Fredkin gates only. This is an auxiliary contribution of this work and may be of independent interest, as we were unable to find any prior implementation or pre-existing parser or circuit converter in the literature or online.

A. OVERVIEW OF IMPLEMENTED FUNCTIONS

The implementation (detailed with listings in the appendix; Section) follows the ProSt protocol defined in Section V, involving:

- 1) **Input Embedding (Section V-A):** Secret bits are embedded into cover images at a randomly chosen secret position using least significant bit (LSB) substitution. Its implementation is given in Section D.
- 2) **Circuit Specification (Section V-B):** A Boolean circuit is defined with inputs (e.g., A, B, C) and gates (AND, OR, NOT), which are implemented using Fredkin gates. The circuit is specifiable by a simple syntax, given hereafter in extended Backus-Naur and regular expression notation. For compactness, we let the letters be from the alphabet $\Lambda = \{a, b, \dots, z, A, B, \dots, Z\}$, and let alphanumeric characters be from the alphabet $\Sigma = \Lambda \cup \{0, 1, \dots, 9, _ \}$.
 - $\text{Sig} := \Lambda \Sigma^*$ are variable's names, i.e. *signals*, following the usual syntax as in other languages, i.e., need to begin with a letter, followed by any alphanumeric character, including an underline $_$. Examples used in this work are A, B, C, X, Y, ...
 - `"INPUT" Sig ("," Sig)*`: specifies the circuit's input signals with names (here A, B, with the ellipsis $\dots$ denoting any number of inputs allowed).
 - `["AND" | "OR"] Sig, Sig -> Sig`: specifies a binary AND, resp. OR, gate, with two inputs. For example, a boolean gate $Z := X \wedge Y$ would be encoded as `AND X, Y -> Z`.
 - `"NOT" Sig -> Sig` denotes a boolean negation, e.g., $Y := \neg X$ would be written as `NOT X -> Y`.

- `"OUTPUT" Sig` marks the output signal of the circuit, which would be returned by the Ext algorithm. The inputs to this final step (i.e., the picture I'' and location (r, c, k)) are secretly communicated from Alice to Bob (cf. Figure 2).

An example of this specification for a concrete circuit will be given later in Section VII-C. The implementation of the parser is given in Section C.

- 3) **Data Processing in Privacy (Section V-C):** The circuit is evaluated pixel-wise across all images, ensuring uniformity to mask the secret position. The implementation of the data processing is given in Section E.
- 4) **Result Extraction (Section V-D):** The output is extracted from the resulting image at the secret position (implementation shown in Section D).

This process ensures that an honest-but-curious cloud provider (Carol) can perform the computation without discerning the secret bits location or content, as proven secure in Theorem 1.

B. EFFICIENCY AND SCALABILITY ANALYSIS

The dominant computational cost arises from applying gates pixel-wise over images of dimensions $h \times w \times 3$ (height, width, channels). For a circuit with N gates, each gate processes $h \cdot w \cdot 3$ pixels, performing constant-time operations (bit extraction, gate logic, and update). Thus, the complexity per gate upon *sequential* evaluation is $O(h \cdot w)$, yielding a total (sequential) complexity of $O(N \cdot h \cdot w)$, or $O(N)$ if we evaluate all gates, i.e., pixels, *in parallel* (if the hardware permits it). Ancillary tasks such as embedding and extraction ($O(W)$ for W wires) and circuit obfuscation ($O(L \cdot W)$ for specification length L) are negligible in comparison. Space complexity is $O(W \cdot h \cdot w \cdot 3)$, as the implementation requires one image per wire, where $W \geq N + \#(\text{inputs}) + \#(\text{ancillaries})$.

For instance, if a circuit requires 100 logical wires and uses $100 \times (1024 \times 1024 \times 3)$ RGB images (one physical image per wire, for simplicity in the current implementation), the total storage needed for these input images alone would be approximately $100 \times (1024 \times 1024 \times 3 \text{ bytes/image}) \approx 300 \text{ MB}$. More complex circuits or larger images would further increase this, potentially reaching several gigabytes, which could lead to significant cloud storage costs and extended data transfer times.

It is important to clarify that while the demonstration implementation (Section VII-C) allocates a distinct `numpy` array object for each logical wire for pedagogical clarity, the underlying ProSt concepts allow for a far more efficient physical mapping. A single RGB image, for example, intrinsically provides $h \times w \times 3$ distinct LSB positions. These positions can be logically assigned to multiple different circuit wires.

To quantify performance, we measured the runtime of our Python implementation using NumPy on a $128 \times 128 \times 3$ RGB image (49,152 pixels) with a circuit comprising 17 gates and 61 wires. Experiments were conducted on an

TABLE 3. Performance comparison of ProSt implementation vs. Other works.

Work	Method	Image Size (pixels)	Gates/Ops	Time (s)	Hardware
ProSt (This Work)	Fredkin Gate, LSB	128×128×3 (49,152)	17 gates, 61 wires	8.38	Intel i5-1135G7, 16 GB
U-Net (2022) [25]	CNN-based (U-Net)	256×256×3 (196,608)	N/A (CNN layers)	2–5	NVIDIA GTX GPU
OMKHES-MS (2022) [26]	Multikey HE + Stego	512×512×3 (786,432)	N/A (enc. ops)	40–80	Assumed i7, 3.6 GHz
EEALSBMR (2019) [27]	Enhanced LSB	256×256×1 (65,536)	N/A	1.5–3	Assumed i5/i7
FHE DB (2018) [28]	Fully HE	N/A (769 rows)	SQL ops	1020	Multi-core CPU
JPEG Stego (2020) [29]	DCT-based LSB	512×512×3 (786,432)	N/A	4–7	Assumed i5/i7

Notes: ProSt runtime is measured; others are adjusted/estimated for hardware and workload. ProSt uses 17 gates and 61 wires. U-Net leverages GPU acceleration. OMKHES-MS and FHE DB reflect encryption overhead. EEALSBMR and JPEG Stego lack homomorphic computation.

HP EliteBook 840 G8 notebook equipped with an 11th Gen Intel Core i5-1135G7 processor (2.4 GHz base frequency, up to 4.2 GHz turbo, 4 cores, 8 threads) and 16 GB of RAM, running Windows 11 Education 64-bit. The total gate execution time, encompassing all homomorphic computations performed by Carol, was recorded as 8.38 seconds.

To contextualise ProSt's performance, we compare it against recent works in steganography and homomorphic encryption. Table 3 presents the results, with runtimes for other works adjusted or estimated based on their reported complexity, image sizes, and assumed hardware relative to our system.

The ProSt implementation (with a runtime of 8.38 seconds) demonstrates its unique capability to perform homomorphic computations on hidden data. This feature is absent in traditional steganography methods such as EEALSBMR and JPEG Steganography, which achieve lower runtimes (1.5–7 seconds) due to simpler embedding operations without computational overhead. Compared to hybrid approaches like OMKHES-MS, ProSt is faster when normalised for image size (e.g., 2.5–5 seconds for 49,152 pixels vs. 40–80 seconds for 786,432 pixels), owing to its lighter gate-based computation compared to encryption complexity. However, GPU-accelerated methods like U-Net Steganography outperform ProSt significantly (2–5 seconds for 196,608 pixels), highlighting the advantage of parallel processing over CPU-bound pixel-wise operations.

Scalability remains a challenge: doubling the gate count to 34 would yield approximately 16.5 seconds, while scaling the image to $256 \times 256 \times 3$ (196,608 pixels) would increase runtime to approximately 33 seconds on our system, consistent with the quadratic scaling in $h \cdot w$. Furthermore, the conceptual requirement of a high number of distinct images for many logical wires (which would necessitate a large collection of physical cover images) is mitigated in practice. Instead of requiring one *physical* image per logical wire, multiple circuit wires can be mapped by Alice to different pixel-channel locations (i.e., varying (r, c, k) triplets) within a *single or a small set of* larger cover images. This strategy significantly reduces the physical data transfer and storage overhead, as the total number of required distinct pixel-channel locations ($h \times w \times 3$) can comfortably accommodate a large number of logical wires (W). To enhance efficiency, we propose two optimizations:

(i) GPU acceleration using libraries like CUDA, potentially reducing runtime to 1–2 seconds by parallelizing pixel operations, and (ii) restricting computations to a smaller secret-containing region (e.g., $h' \times w' < h \times w$), though this necessitates secure region selection to avoid compromising security. These strategies could mitigate the quadratic scaling with image size, improving ProSt's practicality for broader applications.

However, reducing the *effective size of the cover images* (i.e., making $h' \cdot w' \cdot 3$ smaller) directly impacts the security parameter n . As per Theorem 1, the adversary's advantage is $\Theta\left(\frac{1}{n}\right)$. A smaller n (from a smaller image size or fixed secret region) would increase the adversary's advantage, thereby weakening the security guarantee. This necessitates a careful trade-off between computational efficiency and the desired level of security, ensuring that n remains sufficiently large to provide cryptographically negligible advantage.

A current limitation, especially apparent in the provided demonstration, is the simplified mapping of one logical wire to one distinct image file. For realistic circuits, this can lead to an impractical number of image files. However, ProSt can be extended to map multiple logical wires to different positions (row, column, channel) within a *single or a small set of* larger cover images. For instance, a single RGB image inherently offers three channels ($k = 0, 1, 2$), and within each channel, multiple pixels can host different logical wires. This logical mapping, managed by Alice, would significantly reduce the *physical* number of images transmitted and stored, mitigating this scalability concern while maintaining security as each secret location remains distinct and unknown to Carol.

C. DEMONSTRATION IMPLEMENTATION

The Python implementation of ProSt provided here exemplifies the system's ability to perform private computations on steganographically hidden data. The code is structured into modular components that correspond to the formal ProSt algorithms defined in Section IV-A: LocGen for secret position selection, Emb for embedding secret bits, Comp for homomorphic computation using Fredkin gates, and Ext for extracting the result. Additional features, such as circuit obfuscation and randomized gate execution, enhance security against an honest-but-curious adversary, as analysed in Section VI. The appendix (section) integrates key code

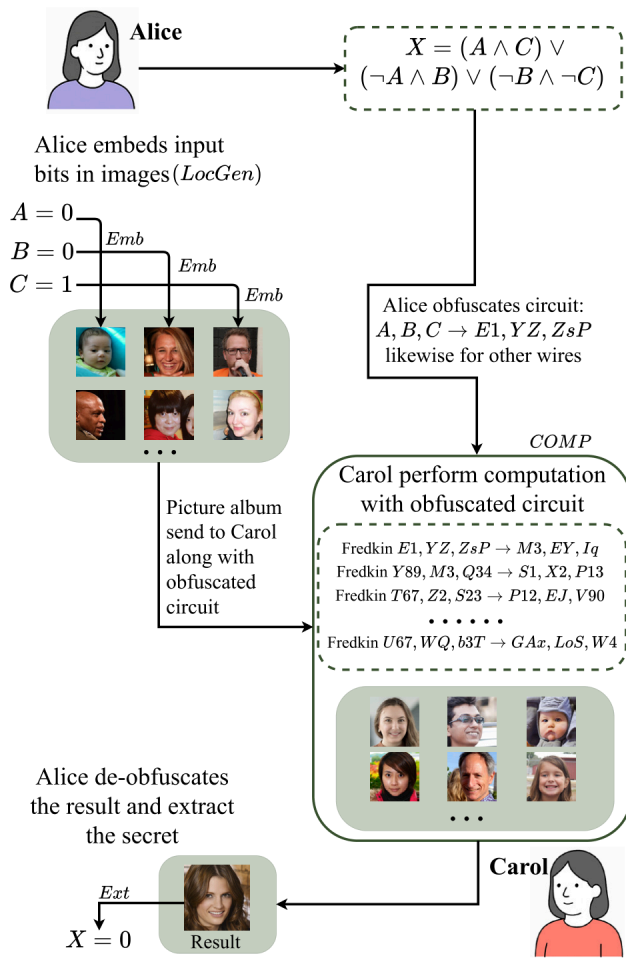


FIGURE 4. Example workflow of computation with homomorphic steganography.

snippets, explains their roles, and relates them to the theoretical foundations presented earlier in the paper. The full implementation, leveraging NumPy and PIL for efficient image handling, will be released upon acceptance of the paper.

The overall workflow (see Section F) demonstrates obfuscation, embedding, computation, and extraction, all summarized in Figure 4. It displays the original pictures into which we embedded the signals and circuit description as the Fredkin-gate's control signals. Since the computation is done on each and every of the least significant bits, the pictures do not visually change over the computation (hence there is no point in repeating them in the paper), but we included them for download along with the source code of our implementation.

VIII. CONCLUSION

This paper introduced Provably Secure Homomorphic Steganography (ProSt), a novel paradigm for secure computation outsourcing in untrusted cloud environments.

By integrating steganographic principles with homomorphic computation via the reversible Fredkin gate, ProSt enables a cloud customer to execute arbitrary Boolean functions on sensitive data while concealing the data's location, its value, and *heuristically obscuring* the computational logic itself from the cloud provider.

Our approach is designed for the practical, honest-but-curious cloud provider model. The system's security is rooted in information-theoretic principles, ensuring that an adversary's advantage in locating secret data diminishes as $\Theta(1/n)$ with the number of decoy bits n . For practical security against honest-but-curious adversaries, we recommend configurations where $n \geq 10^6$ (achievable with high-resolution images or multiple medium-resolution images), yielding adversary advantage below 10^{-6} .

Crucially, ProSt offers two unique advantages in a cloud context. First, as a purely algorithmic solution, it is hardware-agnostic, providing a valuable alternative to hardware-based approaches like Trusted Execution Environments (TEEs) and avoiding dependencies on specific CPU architectures. Second, it *heuristically* hides the computation's function, making it difficult for a provider to reverse-engineer a client's proprietary algorithm, a feature not typically offered by standard homomorphic encryption. However, we emphasize that formal *circuit privacy guarantees* remain an *open problem* for future work.

The practical implementation demonstrated ProSt's feasibility for evaluating Boolean circuits, with clear applications in privacy-preserving analytics, confidential image processing, and general private function evaluation. By avoiding server-side key management and intermediate decryption steps, ProSt reduces the attack surface common in other secure outsourcing frameworks. It represents a new point in the design space of secure cloud computing, trading data transfer overhead for algorithmic simplicity and resilience against certain physical and microarchitectural side-channel attacks.

Future theoretical work will concern the formalisation and analysis of how well the system protects the circuit from reverse engineering (not in terms of topology, but in terms of which gates are used), and extensions also to hide the circuit topology (if possible). This would involve adapting frameworks from homomorphic encryption for circuit privacy to the steganographic domain.

Future practical work can focus on optimising computational efficiency to make ProSt practical for larger, more complex cloud workloads; exploring GPU acceleration is a promising direction. Extending ProSt to other media (e.g., audio, video streams) and investigating alternative embedding techniques could broaden its applicability. Finally, addressing scalability challenges related to data transfer and enhancing resistance against more powerful adversaries, such as active attackers, will be critical for maturing ProSt into a cornerstone technology for secure and covert data processing in the modern cloud.

APPENDIX A

The Appendices describe the Python code (<https://github.com/shahzadssg/prost.git>) for randomized gate ordering and obfuscation functions.

A. CODE OVERVIEW AND STRUCTURE

The code for the ProSt system is structured into several integral sections, each designed to address a distinct facet of its functionality. It begins with the implementation of gate definitions, incorporating the Fredkin gate, which serve as the foundation for reversible and universal computations within the system. This is followed by the circuit specification, where a dedicated `Circuit` class and associated parsing logic are established to interpret Boolean circuit specifications accurately. To enhance security, the system includes a randomized gate ordering mechanism, which ensures that gate execution occurs in a secure manner by randomizing the sequence while preserving necessary dependencies. Further bolstering security, obfuscation functions are employed to rename wires and introduce dummy components into the circuit, thereby concealing its structure. The steganography components manage critical tasks such as generating secret positions, embedding bits, and facilitating their extraction. Additionally, the cloud computation section enables homomorphic computation to be performed on an untrusted cloud provider, referred to as Carol. Finally, the main workflow orchestrates the entire ProSt process, seamlessly integrating input embedding through to result extraction. Below, key snippets are presented and explained to demonstrate how these components collectively realize the ProSt protocol.

B. FREDKIN GATE IMPLEMENTATION

The Fredkin gate, central to ProSt's reversibility and universality, is implemented as follows:

```
1 def fredkin_gate(c, x, y):
2     """Implements the Fredkin gate: swaps x and y if c=1."""
3     if c == 0:
4         return (c, x, y)
5     else:
6         return (c, y, x)
```

This matches the formal definition in Section III-A3, where $F(c, x, y) = (c, x, y)$ if $c = 0$, and (c, y, x) if $c = 1$. The gate is extended to operate pixel-wise across entire images:

```
1 def apply_fredkin_pixelwise(input_images, output_images):
2     """Apply Fredkin gate (text(pixel-wise)) to three input images and set three
3     output images."""
4     I_c, I1, I2 = input_images
5     O_c, O1, O2 = output_images
6     h, w, c = I_c.shape
7     for r in range(h):
8         for col in range(w):
9             for ch in range(c):
10                c_bit = I_c[r, col, ch] & 1
11                x_bit = I1[r, col, ch] & 1
12                y_bit = I2[r, col, ch] & 1
13                c_out, x_out, y_out = fredkin_gate(c_bit, x_bit, y_bit)
14                O_c[r, col, ch] = (O_c[r, col, ch] & ~1) | c_out
15                O1[r, col, ch] = (O1[r, col, ch] & ~1) | x_out
16                O2[r, col, ch] = (O2[r, col, ch] & ~1) | y_out
```

This function applies the Fredkin gate to the least significant bit (LSB) of each pixel across three input images (I_c, I_1, I_2), producing three output images (O_c, O_1, O_2). The uniform application across all pixels, as required by the Comp

algorithm (Section IV-A), ensures that the secret position remains indistinguishable, supporting the security guarantees of Theorem 1.

C. CIRCUIT SPECIFICATION AND PARSING

The code defines a `Circuit` class to represent the Boolean circuit and a `parse_circuit` function to interpret its specification:

```
1 class Circuit:
2     def __init__(self):
3         self.inputs = []
4         self.ancillaries = [] # List of (wire, value)
5         self.gates = []
6         self.outputs = []
7
8     def parse_circuit(spec):
9         """Parse circuit specification with multiple gate types and ancillary
10        wires."""
11        circuit = Circuit()
12        for line in spec.splitlines():
13            line = line.strip()
14            if not line or line.startswith('#'):
15                continue
16            tokens = line.split()
17            if tokens[0] == 'INPUT':
18                for name in tokens[1:].split(','):
19                    circuit.add_input(name.strip())
20            elif tokens[0] == 'ANCILLARY':
21                for part in tokens[1:].split(','):
22                    wire, value = part.split('=')
23                    circuit.add_ancillary(wire.strip(), int(value.strip()))
24            elif tokens[0] in GATE_TYPES:
25                gate_type = tokens[0]
26                expected_io = GATE_TYPES[gate_type]
27                arrow_idx = tokens.index('->')
28                inputs = [x.strip() for x in tokens[1:arrow_idx].split(',')]
29                outputs = [x.strip() for x in tokens[arrow_idx + 1:].split(',')]
30                circuit.add_gate(gate_type, inputs, outputs)
31            elif tokens[0] == 'OUTPUT':
32                for name in tokens[1:].split(','):
33                    circuit.add_output(name.strip())
34        return circuit
```

This supports a circuit specification that includes inputs, ancillary wires (e.g., constants like `const0=0`), Fredkin gate, and outputs. The parser enables Alice to define a circuit, such as $(A \wedge C) \vee (\neg A \wedge B) \vee (\neg B \wedge \neg C)$, which is then converted into a Fredkin-gate-based representation for homomorphic evaluation.

D. STEGANOGRAPHY COMPONENTS

The steganography components implement LocGen, Emb, and Ext:

```
1 def locgen(image_shape):
2     """Alice provides the secret position via user input."""
3     h, w, c = image_shape
4     r, col, ch = map(int, input(f'Enter secret position as row,column,channel' (
5         max row {h-1}, col {w-1}, channel {c-1}): ').split(','))
6     return (r, col, ch)
7
8 def embed_bit_in_image(image_array, bit, pos):
9     """Alice embeds a bit into an image."""
10    r, col, ch = pos
11    pixel = image_array[r, col, ch]
12    image_array[r, col, ch] = (pixel & ~1) | bit
13    return image_array
14
15 def extract_bit_from_image(image_array, pos):
16     """Alice extracts a bit from an image."""
17    r, col, ch = pos
18    bit = image_array[r, col, ch] & 1
19    return bit
```

- **LocGen:** Prompts Alice for a secret position (r, c, k) , ensuring it remains confidential (Section IV-A.i).
- **Emb:** Embeds a secret bit into an image at (r, c, k) using LSB substitution (Section IV-A.ii).
- **Ext:** Extracts the result from the output image at (r, c, k) (Section IV-A.iv).

These functions use LSB steganography, maintaining imperceptibility while enabling computation.

E. CLOUD COMPUTATION

Carol's computation is handled by:

```
1 def Carol_compute_circuit(circuit_spec, received_images, cover_dir):
2     """Carol performs homomorphic computation with randomized gate ordering."""
3     circuit = parse_circuit(circuit_spec)
4     gate_order = randomized_topological_sort(circuit.gates)
5     signals = received_images.copy()
6     for gate_idx in gate_order:
7         gate = circuit.gates[gate_idx]
8         input_images = [signals[wire] for wire in gate['inputs']]
9         output_images = [signals[wire] for wire in gate['outputs']]
10        if gate['type'] == 'FREDKIN':
11            apply_fredkin_pixelwise(input_images, output_images)
12    return signals
```

This implements the Comp algorithm (Section IV-A.iii), applying gates pixel-wise across all images in a randomized order, ensuring correctness without revealing the secret position.

F. MAIN WORKFLOW

The main function ties everything together:

```
1 def main():
2     circuit_spec = """
3     INPUT A, B, C
4     ANCILLARY const0 = 0
5     ANCILLARY const1 = 1
6     FREDKIN A, C, const0 -> A1, C1, AC~ # \ldots (additional gates for (A AND
7     C) OR (NOT A AND B) OR (NOT B AND NOT C))
8     OUTPUT result
9     """
10    obfuscated_spec, mapping = obfuscate_circuit_spec(circuit_spec,
11    num_dummy_ancillaries=5, num_dummy_gates=3)
12    circuit = parse_circuit(obfuscated_spec)
13    images_to_send = [wire: np.array(Image.open(img_path).convert('RGB')) for
14    wire, img_path in zip(all_wires, image_files)]
15    secret_pos = locgen(image_shape)
16    for wire in circuit.inputs:
17        val = int(input(f'Enter bit for {wire} (0 or 1): '))
18        embed_bit_in_image(images_to_send[wire], val, secret_pos)
19    for wire, value in circuit.ancillaries:
20        embed_bit_in_image(images_to_send[wire], value, secret_pos)
21    result_images = Carol_compute_circuit(obfuscated_spec, images_to_send,
22    cover_dir)
23    result = extract_bit_from_image(result_images[circuit.outputs[0]], secret_pos)
24    print(f'Final circuit output: {result}')
```

G. RANDOMIZED GATE ORDERING

To enhance security, the code randomizes the gate execution order while respecting dependencies:

```
1 def randomized_topological_sort(gates):
2     """Generate a randomized execution order for gates while respecting
3     dependencies."""
4     graph, in_degree, sources = build_dependency_graph(gates)
5     order = []
6     while sources:
7         gate_idx = random.choice(sources)
8         order.append(gate_idx)
9         sources.remove(gate_idx)
10        for dependent in graph[gate_idx]:
11            in_degree[dependent] -= 1
12            if in_degree[dependent] == 0:
13                sources.append(dependent)
14    return order
```

This function ensures that Carol evaluates gates in a topologically valid order, reducing the risk of inferring the circuit's structure, as will be discussed in more detail in Section VI-E on partial circuit hiding.

H. OBFUSCATION FUNCTIONS

Security is further strengthened by obfuscating the circuit specification:

```
1 def obfuscate_circuit_spec(circuit_spec, num_dummy_ancillaries=5,
2    num_dummy_gates=3):
3     """Obfuscate the circuit specification with random names and dummy
4     components."""
5     all_wires = get_all_wires(circuit_spec)
6     used_names = set()
7     mapping = {}
8     for wire in all_wires:
9         random_name = generate_random_name(used_names)
10        mapping[wire] = random_name
11        used_names.add(random_name)
```

```
10 obfuscated_spec = circuit_spec
11 for original, random_name in mapping.items():
12     obfuscated_spec = re.sub(r'\b' + re.escape(original) + r'\b', random_name,
13     obfuscated_spec)
14 # Add dummy ancillaries and gates \ldots
15 return obfuscated_spec, mapping
```

This function renames all wires and adds dummy ancillaries and gates, making it harder for Carol to deduce the original computation, aligning with the security enhancements noted in Section VI-E.

REFERENCES

- [1] C. Gentry. (2009). *Fully Homomorphic Encryption Using Ideal Lattices*. [Online]. Available: <https://www.semanticscholar.org/paper/6e6e67042c647b8dd6818f893cbe1ffd474636a3>
- [2] V. Costan and S. Devadas. (2016). *Intel SGX Explained*. [Online]. Available: <https://eprint.iacr.org/2016/086>
- [3] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge, U.K.: Cambridge Univ. Press, 2009. [Online]. Available: <https://www.cambridge.org/core/books/steganography-in-digital-media/2D5989F2462923428BE01F2042A532AB>
- [4] V. Holub and J. Fridrich, "Designing steganographic distortion using directional filters," in *Proc. IEEE Int. Workshop Inf. Forensics Secur. (WIFS)*, Dec. 2012, pp. 234–239.
- [5] T. Pevný, T. Filler, and P. Bas, "Using high-dimensional image models to perform highly undetectable steganography," in *Information Hiding*. Cham, Switzerland: Springer, 2010, pp. 161–177.
- [6] B. Li, M. Wang, J. Huang, and X. Li, "A new cost function for spatial image steganography," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2014, pp. 4206–4210.
- [7] G. Kaptchuk, T. M. Jois, M. Green, and A. Rubin, "Meteor: Cryptographically secure steganography for realistic distributions," Tech. Rep., 2021. [Online]. Available: <https://eprint.iacr.org/2021/686>
- [8] J. Ding, K. Chen, Y. Wang, N. Zhao, W. Zhang, and N. Yu, "Discop: Provably secure steganography in practice based on 'distribution copies,'" in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2238–2255.
- [9] X. Zhang, K. Chen, N. Zhao, W. Zhang, and N. Yu, "Provably secure public-key steganography based on admissible encoding," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3161–3175, 2025.
- [10] Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) LWE," Tech. Rep., 2011. [Online]. Available: <https://eprint.iacr.org/2011/344>
- [11] M. Naehrig, K. Lauter, and V. Vaikuntanathan, "Can homomorphic encryption be practical?" in *Proc. 3rd ACM Workshop Cloud Comput. Secur. Workshop*. New York, NY, USA: Association for Computing Machinery, Oct. 2011, pp. 113–124, doi: [10.1145/2046660.2046682](https://doi.org/10.1145/2046660.2046682).
- [12] C. Gentry, S. Halevi, and N. P. Smart, "Homomorphic evaluation of the AES circuit," in *Proc. Annu. Cryptol. Conf.*, 2012. [Online]. Available: <https://eprint.iacr.org/2012/099>
- [13] K. S. Perumalla, *Introduction to Reversible Computing*. London, U.K.: Chapman & Hall, Sep. 2013.
- [14] M. Saeedi and I. L. Markov, "Synthesis and optimization of reversible circuits—A survey," *ACM Comput. Surv.*, vol. 45, no. 2, pp. 1–34, Mar. 2013, doi: [10.1145/2431211.2431220](https://doi.org/10.1145/2431211.2431220).
- [15] Y. Lindell, "How to simulate it—A tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*, 2016. [Online]. Available: <https://eprint.iacr.org/2016/046>
- [16] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 2nd ed., London, U.K.: Chapman & Hall, 2014.
- [17] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, Aug. 2014. [Online]. Available: <https://www.nowpublishers.com/article/Details/TCS-042>
- [18] D. Boneh, C. Gentry, S. Halevi, F. Wang, and D. J. Wu, "Private database queries using somewhat homomorphic encryption," in *Applied Cryptography and Network Security*, M. Jacobson, M. Locasto, P. Mohassel, and R. Safavi-Naini, Eds., Berlin, Germany: Springer, 2013, pp. 102–118.
- [19] C. Cachin, "Entropy measures and unconditional security in cryptography," Ph.D. dissertation, ETH Zurich, 1997, vol. 1.
- [20] C. Cachin. (2000). *An Information-Theoretic Model for Steganography*. [Online]. Available: <https://eprint.iacr.org/2000/028>

- [21] Y. Dodis, "Shannon impossibility, revisited," in *Information Theoretic Security*. Berlin, Germany: Springer, 2012, pp. 100–110.
- [22] A. Y. AlKhamese, W. R. Shabana, and I. M. Hanafy, "Data security in cloud computing using steganography: A review," in *Proc. Int. Conf. Innov. Trends Comput. Eng. (ITCE)*, Feb. 2019, pp. 549–558.
- [23] S. Vaidya. (Mar. 2023). *OpenStego*. [Online]. Available: <https://github.com/syvaidya/openstego>
- [24] O. Goldreich, "Foundations of cryptography. II: Basic applications," Tech. Rep., May 2004.
- [25] V. Himthani, V. S. Dhaka, M. Kaur, G. Rani, M. Oza, and H.-N. Lee, "Comparative performance assessment of deep learning based image steganography techniques," *Sci. Rep.*, vol. 12, no. 1, p. 16895, Oct. 2022. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9546933/>
- [26] I. Abunadi, H. A. Mengash, S. S. Alotaibi, M. M. Asiri, M. A. Hamza, A. S. Zamani, A. Motwakel, and I. Yaseen, "Optimal multikey homomorphic encryption with steganography approach for multimedia security in Internet of Everything environment," *Appl. Sci.*, vol. 12, no. 8, p. 4026, Apr. 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/8/4026>
- [27] D. Battikh, S. El Assad, T. M. Hoang, B. Bakhache, O. Deforges, and M. Khalil, "Comparative study of three steganographic methods using a chaotic system and their universal steganalysis based on three feature vectors," *Entropy*, vol. 21, no. 8, p. 748, Jul. 2019, doi: [10.3390/e21080748](https://doi.org/10.3390/e21080748).
- [28] A. Chatterjee and I. Sengupta, "Translating algorithms to handle fully homomorphic encrypted data on the cloud," *IEEE Trans. Cloud Comput.*, vol. 6, no. 1, pp. 287–300, Jan. 2018.
- [29] A. Almohammad, G. Ghinea, and R. Hierons, *JPEG Steganography: A Performance Evaluation of Quantization Tables*, Jan. 2009.



interests include plausible deniability, cloud security, and signal processing.



with applications in system security, especially robotics security, complexity theory, statistics, and information-theoretic security. He participated in various nationally and internationally funded research projects, and being a contributing researcher in many EU projects and offering consultancy services to the industry.

...